BRNO UNIVERSITY OF TECHNOLOGY
VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ

FACULTY OF INFORMATION TECHNOLOGY
FAKULTA INFORMAČNÍCH TECHNOLOGIÍ

DEPARTMENT OF COMPUTER SYSTEMS
ÚSTAV POČÍTAČOVÝCH SYSTÉMŮ

3D NEURAL CELLULAR AUTOMATA

3D NEURÁLNÍ CELULÁRNÍ AUTOMATY

BACHELOR'S THESIS
BAKALÁŘSKÁ PRÁCE

AUTHOR **MICHAL REPČÍK**
AUTOR PRÁCE

SUPERVISOR **Ing. KAREL FRITZ,**
VEDOUCÍ PRÁCE

BRNO 2026

Abstract

Do tohoto odstavce bude zapsán výtah (abstrakt) práce v anglickém jazyce.

Abstrakt

Do tohoto odstavce bude zapsán výtah (abstrakt) práce v českém (slovenském) jazyce.

Keywords

Sem budou zapsána jednotlivá klíčová slova v anglickém jazyce, oddělená čárkami.

Klíčová slova

Sem budou zapsána jednotlivá klíčová slova v českém (slovenském) jazyce, oddělená čárkami.

Reference

REPČÍK, Michal. *3D Neural Cellular Automata*. Brno, 2026. Bachelor's thesis. Brno University of Technology, Faculty of Information Technology. Supervisor Ing. Karel Fritz,

3D Neural Cellular Automata

Declaration

I hereby declare that this Bachelor's thesis was prepared as an original work by the author under the supervision of Ing. Karel Fritz I have listed all the literary sources, publications and other sources, which were used during the preparation of this thesis. I have used ChatGPT to correct spelling and other language mistakes. I used Grok when working on the software.

. . . . . .
Michal Repčík
November 29, 2025

Acknowledgements

I would like to express my sincere thanks to my supervisor, Ing. Karel Fritz, for his guidance and support.

Contents

1 Introduction **3**

2 Abstract **4**

3 Theoretical Foundations **5**

3.1 Prerequisites 5

3.2 Cellular Automata 5

3.3 Neural Cellular Automata 6

3.4 Core Components of a 3D Neural Cellular Automaton 7

4 Releated Work **9**

5 Implementation **10**

6 Conclusion **11**

Bibliography **12**

List of Figures

Chapter 1

Introduction

TODO

Chapter 2

Abstract

TODO

Chapter 3

Theoretical Foundations

3.1 Prerequisites

This thesis assumes familiarity with basic concepts of deep learning, including convolutional neural networks (CNNs), gradient-based optimisation, and automatic differentiation. Standard references include Goodfellow et al. [6] and the PyTorch documentation [13, 3].

The following sections provide all necessary background on cellular automata and their neural extensions.

3.2 Cellular Automata

Cellular automata (CA) are discrete dynamical systems composed of a regular grid of cells, each assigned a state from a finite set. The state of each cell is updated synchronously in discrete time steps using a local transition rule that depends only on the states of neighbouring cells [10].

A cellular automaton is defined by:

- a **lattice** of cells (typically 1D, 2D, or 3D),
- a **finite state set** $\mathcal{S}$,
- a **neighbourhood function** $\mathcal{N}$,
- a local **update rule** $f : \mathcal{S}^{|\mathcal{N}|} \to \mathcal{S}$ [8].

The concept was introduced by Ulam and von Neumann [12] and later systematised by Wolfram [18, 19], who identified four qualitative behavioural classes: uniform, periodic, chaotic, and complex. These behavioural categories remain relevant in the analysis of neural cellular automata, which often exhibit similar transitions between order and chaos.

3.2.1 One-Dimensional Cellular Automata

One-dimensional CA use a linear array of cells with neighbourhoods defined by a fixed radius. Elementary CA, introduced by Wolfram [18], use radius 1 and two binary states, yielding 256 possible rules.

Classical examples include:

- Rule 90 producing fractal Sierpiński patterns,

- Rule 30 generating chaotic, pseudorandom behaviour,
- Rule 110, proven to be Turing-complete [4].

3.2.2 Two-Dimensional Cellular Automata

In 2D CA, the most common neighbourhood is the Moore neighbourhood, defined by the 3×3 block centred on each cell. Because the number of possible input configurations is large, rules are often encoded using birth/survival notations.

Conway's Game of Life [5] is the canonical example. Despite its simple update rule, it supports still lifes, oscillators, spaceships, and complex computational structures. Life is Turing-complete [1] and remains a central reference point in CA research.

3.2.3 Three-Dimensional Cellular Automata

Three-dimensional CA extend the neighbourhood to a $3 \times 3 \times 3$ cube, containing up to 26 neighbouring cells. The huge number of possible binary neighbourhood configurations (2^{27}) makes explicit rule specification impractical; therefore, most 3D CA use totalistic rules depending only on the count of active neighbours [19].

Notable examples include von Neumann's 29-state self-replicating automaton [12], multi-state excitable media such as Brian's Brain [9], and several 3D Life-like rules. Advances in hardware accelerated interest in 3D CA simulations in the 2000s [17], and their volumetric nature aligns naturally with later neural extensions.

3.2.4 Continuous and Differentiable Cellular Automata

Continuous extensions of CA replace discrete states with real-valued fields and use smooth update operators. These systems are not trained by gradient descent but illustrate how discrete local rules can generalise to continuous domains.

SmoothLife [14] introduced smooth neighbourhood averages and continuous transition functions producing glider-like dynamics. Lenia [2] extended this approach using parametric kernels and growth functions, revealing a wide range of stable „lifeforms“ in continuous space and time.

Although these models are not neural networks, they demonstrate that local, differentiable update rules can support complex self-organising behaviour. This idea later influenced neural cellular automata.

3.3 Neural Cellular Automata

Neural Cellular Automata (NCA) replace fixed transition rules with a shared, learnable per-cell update function. The grid structure remains discrete, but each cell stores a vector of real-valued channels, making the entire system differentiable and trainable using gradient-based methods.

The foundational work is the "Growing Neural Cellular Automata" model by Mordvintsev et al. [11]. Their model demonstrated self-organising behaviours such as pattern growth, persistence, and regeneration. The update rule is implemented as a small convolutional neural network operating locally on each cell.

Follow-up research explored memory mechanisms [15], multicellular classification [7], and applications to differentiable morphogenesis and pattern formation. Most work to date remains restricted to two dimensions.

Prototype 3D extensions exist, such as the voxel-based implementation by Rossi and Orsini [16], which adapts the 2D model to volumetric grids. These early implementations lack modularity but illustrate the feasibility of stable 3D neural CA.

3.4 Core Components of a 3D Neural Cellular Automaton

The following subsections summarise the theoretical components common to three-dimensional neural cellular automata. These concepts apply broadly and are not tied to any specific implementation.

3.4.1 State Representation

Each cell in a neural CA stores a real-valued state vector $\mathbf{s} \in \mathbb{R}^C$ with C channels. This multi-channel design, introduced in [11], separates:

- **visible channels**, representing observable quantities compared to the training target (e.g., occupancy),
- **hidden channels**, which serve as internal memory and support long-range coordination.

A dedicated *alive* channel provides a continuous gating signal instead of a binary alive/dead flag. Continuous state representations allow gradients to propagate through the update rule, enabling end-to-end differentiability.

3.4.2 Perception Mechanism

Although neural CA operate on continuous states, they retain the fundamental locality of classical CA: each cell updates based on information drawn from a small neighbourhood [19, 8].

Modern NCA implement this locality using a fixed *perception stage* introduced by Mordvintsev et al. [11]. A predefined set of convolutional filters (e.g., 3×3 in 2D, $3 \times 3 \times 3$ in 3D) is applied to each channel to extract simple spatial features. These filters are not learned; they provide structured input to the update rule and improve training stability [11, 15].

Typical filter groups include:

- **identity filters**, returning the central cell state,
- **neighbour aggregations**, summarising adjacent cells,
- **gradient filters**, including Sobel-like operators that approximate spatial derivatives.

The outputs are concatenated into a *perceived state* vector that serves as input to the update function.

3.4.3 Update Rule

The update rule is a shared, parameterised function applied independently to each cell. In most NCA models, it takes the form of a small multi-layer perceptron or a stack of 1×1 convolutions [11]. The function outputs a state increment, which is added to the current state:

$$\mathbf{s}_{t+1} = \mathbf{s}_t + m_t \cdot \Delta\mathbf{s},$$

where m_t is an *alive mask* derived from the alive channel. This gating mechanism restricts updates to active cells and helps stabilise long rollouts.

Stochastic update dropout, sometimes implemented as a probabilistic "fire rate", can be used to improve robustness but is not an inherent property of NCA [11].

3.4.4 Training and Optimisation

Neural CA are trained via gradient descent by comparing visible channels against a target pattern. Training involves unrolling the automaton for multiple steps and backpropagating through the sequence [11]. Different training regimes encourage different behaviours, such as growth, persistence, or regeneration.

Loss functions typically operate on the visible channels, while hidden channels remain unconstrained. Stabilisation techniques include:

- stochastic update dropout,
- random damage injection,
- multi-step rollout training.

These methods support the emergence of self-organising structures and have been applied in both 2D and early 3D models [16].

Chapter 4

Releated Work

TODO

Chapter 5

Implementation

TODO

Chapter 6

Conclusion

TODO

Bibliography

[1] Berlekamp, E. R.; Conway, J. H. and Guy, R. K. *Winning Ways for Your Mathematical Plays, Volume 2*. Academic Press, 1982.

[2] Chan, B. W.-C. *Lenia: Biology of Artificial Life*. 2019.

[3] Community, P. *PyTorch Documentation*. 2025. Available at: https://docs.pytorch.org/stable/index.html.

[4] Cook, M. *Universality in Elementary Cellular Automata*. 2004.

[5] Gardner, M. *Mathematical Games: The fantastic combinations of John Conway's new solitaire game "Life"*. 1970. Available at: https://web.stanford.edu/class/sts145/Library/life.pdf.

[6] Goodfellow, I.; Bengio, Y. and Courville, A. *Deep Learning*. MIT Press, 2016. Available at: https://www.deeplearningbook.org.

[7] Horváth, D. and Levin, M. *Self-Classifying MNCA Distill*. 2021. Available at: https://distill.pub/2021/self-classifying-mnca/.

[8] Ilachinski, A. *Cellular Automata: A Discrete Universe*. World Scientific, 2001. ISBN 981-02-4623-4.

[9] Kinderen, E. de and Wuensche, A. 3D CA in a fluid dynamics PDE. In: *Proceedings of the Second International Conference on Evolvable Systems: From Biology to Hardware*. Springer, 1998.

[10] Marinescu, D. C. Chapter 2 - Nature-Inspired Algorithms and Systems. In: Marinescu, D. C., ed. *Complex Systems and Clouds*. Boston: Elsevier, 2017, p. 33–63. Computer Science Reviews and Trends. ISBN 978-0-12-804041-6. Available at: https://www.sciencedirect.com/science/article/pii/B9780128040416000025.

[11] Mordvintsev, A.; Randazzo, E.; Niklasson, E. and Levin, M. *Growing Neural Cellular Automata*. 2020. Available at: https://distill.pub/2020/growing-ca/.

[12] Neumann, J. von. *Theory of Self-Reproducing Automata*. University of Illinois Press, 1966.

[13] Paszke, A.; Gross, S.; Massa, F.; Lerer, A.; Bradbury, J. et al. *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. 2019. Available at: https://arxiv.org/abs/1912.01703.

[14] Rafler, S. *Generalization of Conway's "Game of Life" to a continuous domain – SmoothLife* arXiv preprint arXiv:1111.1567. 2011. Available at: https://doi.org/10.48550/arXiv.1111.1567.

[15] Randazzo, E.; Mordvintsev, A. and Levin, M. *Persistent Memory in Growing Neural Cellular Automata* Google Research Blog. 2021. Available at: https://ai.googleblog.com/2021/12/persistent-memory-in-neural-cellular.html.

[16] Rossi, F. and Orsini, M. *3D Growing Neural Cellular Automaton*. GitHub, 2021. Available at: https://github.com/SkyLionx/3d-cellular-automaton. Originally published under username SkyLionx.

[17] Tobias, P. A. G. *3D Cellular Automata*. 2009. Available at: https://eprints.bournemouth.ac.uk/17408/.

[18] Wolfram, S. *Statistical Mechanics of Cellular Automata*. 1983.

[19] Wolfram, S. *A New Kind of Science*. Wolfram Media, 2002. ISBN 1-57955-008-8. Available at: https://www.wolframscience.com/nks/.